Python Engineering - Programming Theory Guide

CLASS 14 - File I/O | Persistence, representation and data flow

EX22 introduces a deeper idea than `open()`: the distinction between **in-memory state** and a **persistent representation**.

1. RAM vs persistent storage

Python variables, arrays and objects normally live in process memory while the program is running.

RAM	File
Temporary for process	Persistent
Rich Python objects	Bytes organized by a format
Very fast	Slower than RAM
Lost when process ends	Remains until modified/deleted

A file exists outside the Python process and can be reopened later.

2. File I/O

```
INPUT -> file -> Python
OUTPUT -> Python -> file
```

Saving a trajectory is output; loading it is input.

3. Files do not know Python objects

```
Python float 100.0
 -> formatting -> text '100.000000'
 -> UTF-8 encoding -> bytes on disk
```

Reading reverses this translation.

4. Serialization and parsing

Serialization: convert data into a storable/transmittable representation. **Parsing:** interpret that representation and rebuild meaningful values.

```
[100.0,20.0,0.0]
 -> '100.000000,20.000000,0.000000\n'
 -> split/float
 -> [100.0,20.0,0.0]
```

The same concept appears in JSON, databases, networks and scientific formats.

5. A file object is a resource handle

The object returned by open() is a Python interface to an operating-system resource, not the physical file itself.

6. Context manager

```
ACQUIRE RESOURCE
 -> USE RESOURCE
 -> RELEASE RESOURCE
```

`with` automates that life cycle.

7. File position/cursor

Reading advances a current position. `readline()` consumes one line; subsequent iteration continues from there.

```
header = file.readline()
for line in file:
    # data rows only
```

8. Why a CSV header matters

A row such as 100.0,20.0,0.0 is ambiguous without meaning and units.

```
h_m,v_m_s,t_s
```

Engineering data needs semantics and units, not only numbers.

9. Written precision vs internal precision

```
internal: 0.30000000000000004
written : 0.300000
loaded  : 0.3
```

Formatting to six decimals is a deliberate precision decision. Approximate validation is therefore appropriate.

10. Round-trip integrity

```
A = original
B = load(save(A))
desired: B approximately equals A
```

We test recoverability of information, not merely file creation.

11. Why file_io.py is not a VerticalState method

VerticalState models physics/state. CSV is a persistence choice. Keeping them separate avoids coupling the physical model to a storage format.

Question	Natural responsibility
How does h,v,t evolve?	VerticalState
How is energy computed?	VerticalState
How is trajectory converted to CSV?	file_io.py
Where should output be stored?	main.py/application config

12. Separation of concerns

```
PHYSICS        -> vertical_state.py
ORCHESTRATION  -> main.py
PERSISTENCE    -> file_io.py
FILESYSTEM     -> data/
```

A future switch from CSV to another format should not require rewriting Euler physics.

13. Path vs file contents

Path	Contents
WHERE resource is	WHAT information it stores
Path/string	Text/bytes
data/trajectory.csv	h_m,v_m_s,t_s ...

14. Current Working Directory

Relative paths are normally resolved from the process working directory, which may differ from the script directory. Anchoring to `__file__` avoids a common VS Code source of confusion.

15. 'w' destroys previous contents

Opening an existing file in write mode truncates it before new data is written. Use it deliberately.

16. 'a' and logs

Append mode writes new content at the end while preserving previous contents.

17. A file format is a contract

```
HEADER: h_m,v_m_s,t_s
ROW ORDER: h,v,t
DELIMITER: comma
NUMBERS: decimal text
```

Writer and reader must agree. A silent column-order mismatch can produce physically wrong results.

18. Structural and numerical validation

```
trajectory.shape == loaded.shape
np.allclose(trajectory, loaded)
```

19. Why no try/except yet

Many failures are possible: missing file, permissions, corrupt rows, invalid numeric text. EX22 first teaches the normal data path. The exceptions class will address failure handling explicitly.

20. Engineering connection

Domain	Typical persistent data
GNSS/INS	measurement logs, trajectories, configs
Aircraft dynamics	state/force histories
Propulsion	test-bench measurements
Astrodynamics	ephemerides, state histories
CFD/FEA	large scientific datasets

Final mental model

```
PYTHON DATA
 -> serialize/format
 -> text representation
 -> encode/write
 -> persistent file
 -> read/decode
 -> parse/type conversion
 -> PYTHON DATA
 -> validate
```

Core idea: File I/O is the design of a reliable translation between a program's internal representation and a persistent representation that can be recovered later.